

Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing

Liang Huang, Xu Feng, Cheng Zhang, Liping Qian, Yuan Wu^{*}

College of Information Engineering, Zhejiang University of Technology, Hangzhou, 310023, China

ARTICLE INFO

Keywords:

Mobile edge computing
Joint computation offloading and resource allocation
Deep-Q network

ABSTRACT

The rapid growth of mobile internet services has yielded a variety of computation-intensive applications such as virtual/augmented reality. Mobile Edge Computing (MEC), which enables mobile terminals to offload computation tasks to servers located at the edge of the cellular networks, has been considered as an efficient approach to relieve the heavy computational burdens and realize an efficient computation offloading. Driven by the consequent requirement for proper resource allocations for computation offloading via MEC, in this paper, we propose a Deep-Q Network (DQN) based task offloading and resource allocation algorithm for the MEC. Specifically, we consider a MEC system in which every mobile terminal has multiple tasks offloaded to the edge server and design a joint task offloading decision and bandwidth allocation optimization to minimize the overall offloading cost in terms of energy cost, computation cost, and delay cost. Although the proposed optimization problem is a mixed integer nonlinear programming in nature, we exploit an emerging DQN technique to solve it. Extensive numerical results show that our proposed DQN-based approach can achieve the near-optimal performance.

1. Introduction

The past decade has witnessed an explosive growth in mobile internet applications that consumed a significant amount of computational resources, e.g., online gaming, virtual/augmented reality, real-time media streaming, and so on. Mobile Edge Computing (MEC), which enables mobile terminals to actively offload computation tasks to nearby edge servers, has provided an efficient approach to address the growing computation demands and realize an efficient computation offloading. Exploiting the sufficient computation resources at the edge server, MEC is expected to bring multi-folded benefits, such as reducing the delay in executing the computation tasks and saving energy. As a result, lots of research efforts have been devoted to studying MEC [1, 2].

In practice, offloading computation task via MEC invokes the data transmission via wireless links. If lots of mobile terminals aggressively offload their computation tasks to the edge server, a severe congestion will occur on wireless links, which leads to a significant delay in

executing MEC. Therefore, to exploit computation offloading, we need a joint management for computation offloading and the associated radio resource allocation, which actually have attracted the attention of a lot of researchers. In general, the relevant studies about resource management for MEC can be categorized into two groups, namely, those focusing on the binary computation offloading and those focusing on the partial computation offloading. Specifically, the binary computation offloading means that a mobile user can either completely offload its task to the edge or execute the task locally. For instance, in Ref. [3], exploiting the MEC, the authors formulated a joint optimization of the binary computation offloading decision, resource allocation, and content caching strategy to maximize the total revenue of the network. In Ref. [4], the authors considered a multi-channel scenario and formulated a multi-user binary offloading-decision game to optimize the number of beneficial users via MEC. In Ref. [5], the authors investigated a joint optimization scheme which jointly optimizes the binary offloading decisions of all users and the communication resource allocation, with the objective to

^{*} Corresponding author.

E-mail addresses: lianghuang@zjut.edu.cn (L. Huang), xfeng_zjut@163.com (X. Feng), CZhang_zjut@163.com (C. Zhang), lpqian@zjut.edu.cn (L. Qian), iewuy@zjut.edu.cn (Y. Wu).

<https://doi.org/10.1016/j.dcan.2018.10.003>

Received 31 May 2018; Accepted 18 October 2018

Available online 24 October 2018

2352-8648/© 2018 Chongqing University of Posts and Telecommunications. Production and hosting by Elsevier B.V. on behalf of KeAi. This is an open access article

under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

minimize the overall cost of energy, computation, and delay for all users. In Ref. [6], the authors investigated a joint optimization of radio resource and computation resources for an energy efficient and latency constrained application offloading. In Ref. [12], the authors investigated a single user's optimal binary computation offloading via MEC in the ultra-dense sliced radio access networks. Different from the binary computation offloading, the partial computation offloading means that a mobile user can offload part of its computation task to the edge server, which is more flexible compared with the binary offloading decision. For instance, in Ref. [7], the authors exploited the users' partial computation offloading and investigated the latency-minimization problem for a multi-user MEC system by jointly optimizing the computation and communication resource allocations. In Ref. [8], the authors proposed a partial computation-offloading that incorporates the dynamic voltage scaling. In Ref. [9], the authors also considered the multi-users' partial computation offloading and investigated the resource allocation based on the time division multiple access and the orthogonal frequency-division multiple access, with the objective of minimizing the weighted sum of users' energy consumptions. Our work here belongs to the first group of the studies, namely focusing on the users' binary computation offloading. Many of the above studies intend to solve the complicated joint computation offloading and resource allocation problems, which are in general non-convex and challenging. Therefore, many different techniques (such as the alternative direction method of multipliers in the convex optimization [3], the game-theoretic modeling in Ref. [4], and the non-convex quadratically constrained quadratic program in Ref. [5].) have been adopted. Recently, along with the explosive growth in the artificial intelligence, e.g., AlphaGo from Google [10], exploiting artificial intelligence for resource management in wireless networks has attracted lots of interests [11,12]. In Ref. [11], He and others proposed a deep reinforcement learning approach to investigate the optimal caching and interference alignment in time-varying practical wireless environments. In Ref. [12], Chen et al. considered MEC for a representative mobile user in an ultra-dense sliced radio access network, where multiple Base Stations (BSs) are available to be selected for computation offloading. The authors proposed a Deep Q-Network (DQN) based strategic computation offloading algorithm to derive the optimal policy to maximize the long-term utility performance. In this work, in order to address the challenges in solving the formulated joint task offloading and bandwidth allocation problem (which corresponds to a mixed integer and non-linear programming), we propose a DQN-based approach to achieve the optimal solution. In particular, DQN can be regarded as an advanced reinforcement learning approach which exploits the deep neural network to replace the original Q-table in the process of Q-learning [13–15]. In particular, the introduced deep neural network can facilitate a quick prediction of the Q-values associated with the system-action pairs, which significantly reduces the computation-burden for enumeration in the original Q-learning. By further optimizing the deep neural network (e.g., by using the gradient descent algorithms), one can obtain an accurate prediction of the Q-values (and reduce the prediction error introduced by the neural network) and thus efficiently solve the bellman-ford equation and obtain the optimal policy. Thanks to the aforementioned advantages, DQN has been widely used to solve some complicated engineering problems, e.g., robot control [16], natural language process [17], and gaming [18].

As we will illustrate in the next section, we present a joint task offloading and bandwidth allocation problem for the multi-user computation offloading via MEC. Our objective is to minimize the overall cost, including the overall delay for finishing the users' tasks, the energy consumption, and the edge server's utility cost. The challenge in solving this problem is that it is a mixed binary nonlinear programming problem. Directly enumerating all the possible solutions is computationally prohibitive. Therefore, exploiting the advantage of DQN, we adopt a DQN based approach, in which we model all the possible solutions as state

spaces and the movement among different states as actions. By doing so, we equivalently transform the original non-convex optimization into a reinforcement learning problem and adopt the DQN approach to find the optimal solution. Extensive numerical evaluations show that our proposed algorithm achieves a stable convergence performance and provides a close-to-optimal solution. What's more, the solution offered by our proposed DQN-based approach outperforms the MUMTO algorithm proposed in Ref. [5].

The rest of this paper is organized as follows: Section 2 presents the system model and problem formulation. Our proposed DQN based algorithm is presented in Section 3. We further show the numerical results in Section 4 and conclude this work in Section 5.

2. System model and problem formulation

2.1. System model

In this work, we consider a system model as shown in Fig. 1. A group of N mobile users offload their computation tasks to the edge server via wireless links. Each user has M independent tasks to be completed.¹

To model the task, we use L_{nm}^{in} to denote the input data size of user n 's m th task, i.e., the workload of task m which needs to be transmitted from user n to the edge server. And we use L_{nm}^{out} to denote the output data size of user n 's m th task, i.e., the workload of task m which needs to be transmitted back from the edge server to user n . Meanwhile, each user n can determine whether to offload its m th task to the edge server or not. Then we use the binary variable $x_{nm} = 1$ to denote that user n decides to offload its task m to the edge server, and $x_{nm} = 0$ means that user n decides to execute its task m locally. We'll give a detailed modeling of the computation offloading in the next subsections. Table 1 lists the important notations used in this paper.

2.2. Cost of computation offloading

When users offload their tasks to the edge server, the energy consumption for user n 's task m includes two parts, i.e., the part for data transmission and the part for data reception. In particular, we denote the energy consumption for data transmission as E_{nm}^{t} and the energy consumption for data reception as E_{nm}^{r} . In addition to the energy consumption of mobile users, we also use a similar approach as [5] to model the system utility cost of the edge server. Specifically, when mobile user n offloads its task m to the edge server, the system utility cost is given by

$$D_{nm}^{\text{clo}} = L_{nm}^{\text{in}} + \frac{\varphi_1}{f_{\text{clo}}} + \frac{\varphi_2}{C_{\text{up,max}}} + \frac{\varphi_3}{C_{\text{do,max}}} \quad (1)$$

Parameters $C_{\text{up,max}}$ and $C_{\text{do,max}}$ denote the total uplink bandwidth and the total downlink bandwidth respectively. Parameter f_{clo} is the CPU rate of the edge server. Similar to [5], we set the weighting parameters $\varphi_1 = 10^{18} \text{bit} \times \text{cycle/s}$ and $\varphi_2 = \varphi_3 = 10^{16} \text{bit} \times \text{bps}$. In summary, we denote the total cost for user n to offload its task m to the edge server as

$$E_{nm}^{\text{clo}} = E_{nm}^{\text{t}} + E_{nm}^{\text{r}} + \lambda D_{nm}^{\text{clo}} \quad (2)$$

where λ is the discount factor. Notice that the cost E_{nm}^{clo} includes the energy consumptions for sending and receiving the task and the server's utility cost for executing this task.

Next, we model the delay in computation offloading. Specifically, we use C_n^{up} to denote the allocated bandwidth to user n for transmitting its offloaded task to the edge server and use C_n^{do} to denote the allocated bandwidth to user n for receiving the output data of its task from the edge

¹ Notice that the formulation and proposed scheme are applicable to the case that different mobile users have different numbers of the tasks to be completed.

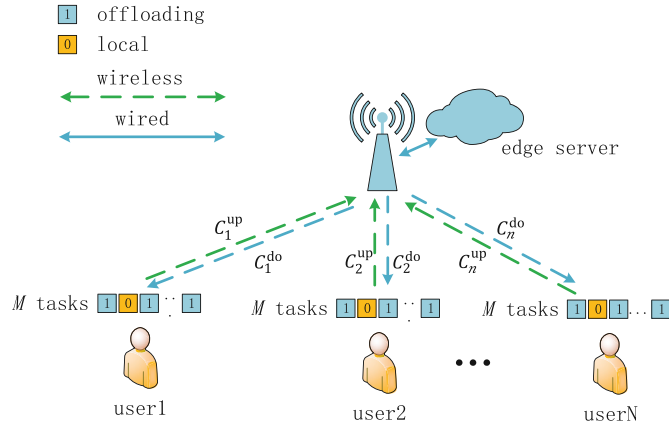


Fig. 1. System model.

Table 1
Important notations used in this paper.

Symbol	Definition
x_{nm}	$x_{nm} = 1$ if user n offloads its task m to the edge server. Otherwise, $x_{nm} = 0$
L_{nm}^{in}	Input data size of user n 's m th task
L_{nm}^{out}	Output data size of user n 's m th task
E_{nm}^{t}	Data transmission energy consumption of task m of user n
E_{nm}^{r}	Data receiving energy consumption of task m of user n
C_n^{up}	Uplink bandwidth assigned to user n
C_n^{do}	Downlink bandwidth assigned to user n
$C_n^{\text{up,max}}$	Total uplink bandwidth
$C_n^{\text{do,max}}$	Total downlink bandwidth
T_{nm}^{up}	Uplink transmission time of task m of user n
T_{nm}^{do}	Downlink transmission time of task m of user n
T_{nm}^{clo}	Edge processing time of task m of user n
N_{nm}^{c}	Number of processing cycles for every input data of task m of user n
f^{clo}	Edge processing rate
D_{nm}^{clo}	System utility cost of task m of user n
T_n^{clo}	Worst-case offloading delay
e_n^{loc}	Energy consumption per data bit
E_{nm}^{loc}	Local energy consumption of task m of user n
t_n^{loc}	Local processing time per data bit
T_{nm}^{loc}	Local processing time consumption of task m of user n
ω_n	Weight between energy consumption and processing delay in the system cost
E_{nm}^{clo}	Energy consumption of the edge server for executing task m of user n
T_n^{loc}	Local processing delay of user n

server. Therefore, the uplink transmission delay when user n offloads its task m to the edge server is given by

$$T_{nm}^{\text{up}} = \frac{L_{nm}^{\text{out}}}{C_n^{\text{up}}} \quad (3)$$

Meanwhile, the transmission delay for user n to receive the data output of its task m is given by

$$T_{nm}^{\text{do}} = \frac{L_{nm}^{\text{in}}}{C_n^{\text{do}}} \quad (4)$$

And the edge processing delay is given by

$$T_{nm}^{\text{clo}} = \frac{L_{nm}^{\text{in}} N_{nm}^{\text{c}}}{f^{\text{clo}}} \quad (5)$$

where f^{clo} is the edge processing rate and N_{nm}^{c} as the number of processing cycles for each bit of input data. In summary, given the offloading decisions $\{x_{nm}\}$, the total delay of user n when it executes MEC can be given by

$$T_n^{\text{clo}} = \sum_{m=1}^M (T_{nm}^{\text{up}} + T_{nm}^{\text{do}} + T_{nm}^{\text{clo}}) x_{nm}, \forall n \quad (6)$$

Similar to [5], in Eq. (6), we consider that the edge-server can only start to process user n 's task m after this task is completely received by the edge, and the edge-server can only start to send back the output data after the entire task m is completed. Thus, all the delays are summed together.

2.3. Cost of local computing

Next we model the case when user n decides to execute its energy locally. Specifically, we use e_n^{loc} to denote the energy consumption per data bit. Thus, user n 's energy consumption for executing its task m locally is given by

$$E_{nm}^{\text{loc}} = L_{nm}^{\text{in}} e_n^{\text{loc}} \quad (7)$$

Meanwhile, we denote user n 's local processing time per data bit as t_n^{loc} . As a result, the total processing time for user n to execute its task m locally is given by

$$T_{nm}^{\text{loc}} = (L_{nm}^{\text{in}} + L_{nm}^{\text{out}}) t_n^{\text{loc}} \quad (8)$$

Given user n 's offloading decision $\{x_{nm}\}$, the total delay for user n to finish its tasks locally is given by

$$T_n^{\text{loc}} = \sum_{m=1}^M T_{nm}^{\text{loc}} (1 - x_{nm}) \quad (9)$$

2.4. Problem formulation

We formulate the optimization problem that aims at minimizing the total delay in finishing all users' tasks and the corresponding energy consumptions, by jointly optimizing each user n 's offloading decisions $\{x_{nm}\}$ and the bandwidth allocations C_n^{up} and C_n^{do} for user n 's task transmission and reception. In particular, based on our previous modeling, user n 's overall delay in completing all its tasks can be given by

$$T_n^{\text{ove}} = \max\{T_n^{\text{loc}}, T_n^{\text{clo}}\} \quad (10)$$

Thus, our formulated Joint Task Offloading and Bandwidth Allocation (JTOBA) can be expressed as follows:

$$(\text{JTOBA}) : \min \sum_{n=1}^N \left[\sum_{m=1}^M (E_{nm}^{\text{loc}} (1 - x_{nm}) + E_{nm}^{\text{clo}} x_{nm}) + \omega_n T_n^{\text{ove}} \right] \quad (11)$$

subject to:

$$\sum_{n=1}^N C_n^{\text{up}} \leq C_n^{\text{up,max}} \quad (12)$$

$$\sum_{n=1}^N C_n^{\text{do}} \leq C_n^{\text{do,max}} \quad (13)$$

$$C_n^{\text{up}}, C_n^{\text{do}} \geq 0, \forall n \quad (14)$$

$$x_{nm} \in \{0, 1\}, \forall n, m \quad (15)$$

variables:

$$\{x_{nm}\}, \{C_n^{\text{up}}\}, \{C_n^{\text{do}}\} \quad (16)$$

Parameter ω_n denotes the weight between energy consumption and the total processing delay. Constraint (12) means that the total uplink bandwidth allocation for all users cannot exceed the maximum bandwidth $C_n^{\text{up},\text{max}}$. Constraint (13) means that the total downlink bandwidth allocation for all users cannot exceed the maximum bandwidth $C_n^{\text{do},\text{max}}$.

We notice that problem (JTOBA) is a typical mixed binary nonlinear programming problem [19], which is generally very challenging.

3. Deep reinforcement learning

3.1. Reinforcement learning

As DQN can be regarded as an advanced Reinforcement Learning (RL), we firstly briefly review RL which is a general framework for decision-making. It is comprised of an environment and an agent with the capacity to act. Each action will gain the agent a scalar reward and possibly influence the agent's future state. The RL algorithm aims at optimally selecting actions to maximize the overall reward in the long-term. With the Markov property of the underlying environment, the RL framework can also be regarded as a Markov decision process [20]. In general, the RL algorithm provides a promising solution to handle the complicated decision-making process.

The Q-learning algorithm, as a typical model-free RL, provides a viable yet well-known methodology for solving the RL problem. Specifically, let π denote the policy. The famous Q function, as a function of the state-action pair, quantifies the long-term accumulative reward which can be given by

$$Q^\pi(s_t, a) = (1 - \alpha)Q(s_t, a) + \alpha \left(r(s_t, a) + \gamma \max_{a'} Q(s_{t+1}, a'; \pi) \right) \quad (17)$$

In Eq. (17), $r(s_t, a)$ denotes the instantaneous reward of choosing action a at state s_t . Parameter γ is the decaying factor, and parameter α is the learning rate. For each state-action pair, Eq. (17) keeps track of a running average of the rewards r and the agent gets upon leaving the state s with action a . In our work, state $s = \{x_{nm}, C_n^{\text{up}}, C_n^{\text{do}}\}$ includes the offloading decisions $\{x_{nm}\}$ and the bandwidth allocations C_n^{up} and C_n^{do} . Action a represents the corresponding change/movement of different states, which is detailed in the next subsection.

However, the complexity in solving Eq. (17) grows exponentially with respect to the size of the state space and that of the action space. Therefore, when the number of the state-action pairs is large, it becomes computational prohibitive to solve Eq. (17) directly. A promising approach to address this difficulty is to approximate the Q-values with a manageable number of parameters (e.g., the conventional Approximate Q-Learning). Recently, with growing advances in DQN, exploiting the DQN to approximate the Q-values and thus to facilitate solving Eq. (17) proves to be an efficient solution methodology.

3.2. Introduction to DQN

In the conventional Q-learning algorithm, Q-table stores the Q-values of all state-action pairs. When the number of state-action pairs is large, it forms a significant computational burden to accurately compute all the Q-values. A practical yet computation-viable approach is to approximate the Q-values of different state-action pairs via prediction, e.g., by using the technique of fitting, which leads to the key essence of

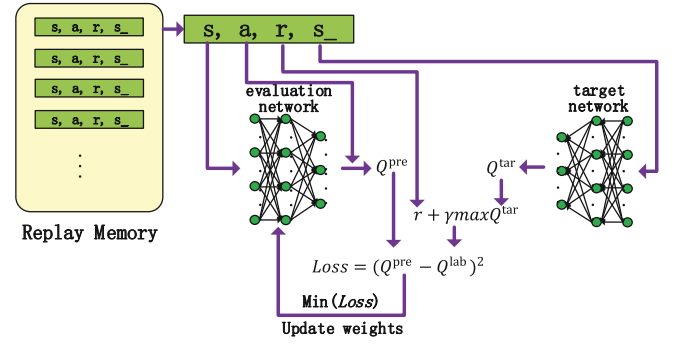


Fig. 2. The relationship between the evaluation network and the target network.

DQN [21,22].

There is a special structure of replay memory to enable prediction via learning in DQN. The purpose of the replay memory is to store a number of records, with each record containing (s_t, a_t, r_t, s_{t+1}) when executing the DQN. In particular, a batch of the memory (e.g., a randomly selected 32 records) is used as the experience for prediction. The system records a memory (s_t, a_t, r_t, s_{t+1}) for every step. During the network training, a batch of memory is extracted from the memory pool, such that the Q network can learn from the previous experience. Thanks to the replay memory, one has sufficient training data for fitting the Q-value for different state-action pairs accurately, which leads to

$$Q^{\text{pre}}(s_t, a, w) \approx Q^*(s_t, a) \quad (18)$$

According to [21] in Eq. (18), the parameter w is a vector used in the deep neural networks for fitting. Fig. 2 illustrates how the prediction in DQN works. Specifically, the evaluation network plays the role of prediction and the target network whose output has been modified by reward will be used as a label of evaluation network as

$$Q^{\text{lab}}(s_t, a) = r(s_t, a) + \gamma \max_{a'} Q^{\text{pre}}(s_{t+1}, a', w') \quad (19)$$

Then, a gradient descent algorithm is applied to minimize the difference between the output of the target network and the prediction (i.e., the output of the evaluation network):

$$\text{Loss} = (Q^{\text{lab}}(s_t, a) - Q^{\text{pre}}(s_t, a, w))^2 \quad (20)$$

In practice, the gradient descent algorithm is exploited to minimize the Loss in (20) and thus update weight w , such that the gap between the predicted value and the target value can be minimized. As a result, the neural network is able to predict with an improved accuracy as the training process continues. Our proposed Algorithm 1 will show the details in the next subsection.

3.3. Exploiting DQN to solve problem (JTOBA)

To solve problem (JTOBA), we exploit the technique of DQN, i.e., to find the optimal offloading decision x_{nm} of task m of each user n as well as the resource allocation C_n^{up} and C_n^{do} of each user n , as illustrated in Fig. 3. Notice that to exploit DQN, we need to specify the state space, the action space, and the reward function, which are illustrated in the following subsections.

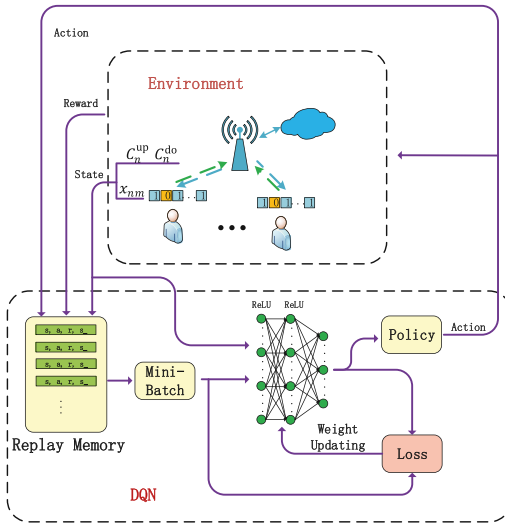


Fig. 3. The flowchart of DQN.

3.3.1. System state

To interpret our problem (JTOBA) as a DQN problem, we define the system state at an arbitrary index t as follows:

$$s(t) = \{x_{11}(t), x_{12}(t), \dots, x_{NM}(t), C_1^{\text{up}}(t), C_1^{\text{do}}(t), \dots, C_N^{\text{up}}(t), C_N^{\text{do}}(t)\} \quad (21)$$

In particular, the system state is a $1 \times (NM + 2N)$ vector, which includes all users' offloading decisions $\{x_{nm}\}$ and the bandwidth allocations $\{C_n^{\text{up}}, C_n^{\text{do}}\}_{n \in \mathcal{J}}$ for all users. Specifically, the offloading decision $x_{nm}(t) \in \{0, 1\}$ for $n = 1, 2, \dots, N, m = 1, 2, \dots, M$. The space for the bandwidth allocation is $C_n^{\text{up}}(t) = \frac{C_n^{\text{up}, \max}}{N^{\text{up}, \text{tot}}} \times c_n^{\text{up}}$, with $c_n^{\text{up}} = \{1, 2, \dots, N^{\text{up}, \text{tot}}\}$, where $N^{\text{up}, \text{tot}}$ means the total number of the choices for the uplink bandwidth allocation. Meanwhile, $C_n^{\text{do}}(t) = \frac{C_n^{\text{do}, \max}}{N^{\text{do}, \text{tot}}} \times c_n^{\text{do}}$, with $c_n^{\text{do}} = \{1, 2, \dots, N^{\text{do}, \text{tot}}\}$, where $N^{\text{do}, \text{tot}}$ denotes the total number of the feasible choices for the bandwidth allocation.

3.3.2. System action

We use the action to denote how we move among two different neighboring system states. Specifically, we define the action as an index-selection (i.e., after the index is selected, the following operation is given, which corresponds to a specific change/movement among two different neighboring states). We use a variable v to denote the selection of index, where $v = 1, 2, \dots, NM + 4N$, and consider the action $a(t) = \{a_v(t)\}$ as an $1 \times (NM + 4N)$ vector. Depending on the selected index v , we have the following cases regarding each element of the action.

Case I: when $1 \leq v \leq NM$, the corresponding action $a_v(t)$ means how we change the offloading decision x_{nm} . Specifically, we use

$$\text{if } a_v(t) = 1, \text{ set } x_{\lfloor \frac{v}{M} \rfloor \bmod(v, M)} = 1 - x_{\lfloor \frac{v}{M} \rfloor \bmod(v, M)} \quad (22)$$

$$\text{if } a_v(t) = 0, \text{ set } x_{\lfloor \frac{v}{M} \rfloor \bmod(v, M)} = x_{\lfloor \frac{v}{M} \rfloor \bmod(v, M)} \quad (23)$$

where $\lfloor \frac{v}{M} \rfloor$ is the integer operation, and $\bmod(v, M)$ is the remainder operation. Then we can find the corresponding user tasks through v .

Case II: when $NM + 1 \leq v \leq NM + N$, the corresponding action $a_v(t)$

means that we increase the uplink bandwidth for the users. Specifically, we use

$$\text{if } a_v(t) = 1, \text{ set } C_{v-NM}^{\text{up}} = C_{v-NM}^{\text{up}} + \Delta C^{\text{up}} \quad (24)$$

$$\text{if } a_v(t) = 0, \text{ set } C_{v-NM}^{\text{up}} = C_{v-NM}^{\text{up}} \quad (25)$$

where the step size for updating the uplink bandwidth is given by $\Delta C^{\text{up}} = \frac{C_n^{\text{up}, \max}}{N^{\text{up}, \text{tot}}}$.

Case III: when $NM + N + 1 \leq v \leq NM + 2N$, the corresponding action $a_v(t)$ means that we decrease the uplink bandwidth for the users. Specifically, we use

$$\text{if } a_v(t) = 1, \text{ set } C_{v-NM-N}^{\text{up}} = C_{v-NM-N}^{\text{up}} - \Delta C^{\text{up}} \quad (26)$$

$$\text{if } a_v(t) = 0, \text{ set } C_{v-NM-N}^{\text{up}} = C_{v-NM-N}^{\text{up}} \quad (27)$$

Case IV: when $NM + 2N + 1 \leq v \leq NM + 3N$, the corresponding action $a_v(t)$ means that we increase the downlink bandwidth for the users. Specifically, we use

$$\text{if } a_v(t) = 1, \text{ set } C_{v-NM-2N}^{\text{do}} = C_{v-NM-2N}^{\text{do}} + \Delta C^{\text{do}} \quad (28)$$

$$\text{if } a_v(t) = 0, \text{ set } C_{v-NM-2N}^{\text{do}} = C_{v-NM-2N}^{\text{do}} \quad (29)$$

where the step size for updating the downlink bandwidth is given by $\Delta C^{\text{do}} = \frac{C_n^{\text{do}, \max}}{N^{\text{do}, \text{tot}}}$.

Case V: when $NM + 3N + 1 \leq v \leq NM + 4N$, the corresponding action $a_v(t)$ means that we decrease the downlink bandwidth for the users. Specifically, we use

$$\text{if } a_v(t) = 1, \text{ set } C_{v-NM-3N}^{\text{do}} = C_{v-NM-3N}^{\text{do}} - \Delta C^{\text{do}} \quad (30)$$

$$\text{if } a_v(t) = 0, \text{ set } C_{v-NM-3N}^{\text{do}} = C_{v-NM-3N}^{\text{do}} \quad (31)$$

3.3.3. Reward function

Recall that at state $s(t)$, after choosing action $a(t)$ based on $\text{argmax}_a Q^{\text{pre}}(s(t), a; w)$, we know the transition of the state from $s(t)$ to $s(t+1)$. In particular, as we have explained before, the state $s(t)$ represents the offloading decisions and the uplink and downlink bandwidth allocations. Therefore, based on $s(t)$, we can derive the value of the objective function in problem (JTOBA). Specifically, we define function $J(t)$ as the value of the objective function in problem (JTOBA). For the sake of easy presentation, we denote

$$J_{s(t)}(t) = JTOBA(\{x_{mn}\}, \{C_n^{\text{up}}, C_n^{\text{do}}\}) \quad (32)$$

where $\{x_{mn}\}$ and $\{C_n^{\text{up}}, C_n^{\text{do}}\}$ are given by the state $s(t)$ according to the definition in Eq. (21). Based on $J_{s(t)}(t)$ and $J_{s(t+1)}(t+1)$, we define the reward of the state-action pair $(s(t), a(t))$ as follows:

$$r_{(s(t), a(t), s(t+1))} = 1, \text{ if } J_{s(t+1)}(t+1) < J_{s(t)}(t) \quad (33)$$

$$r_{(s(t), a(t), s(t+1))} = -1, \text{ if } J_{s(t+1)}(t+1) > J_{s(t)}(t) \quad (34)$$

$$r_{(s(t), a(t), s(t+1))} = 0, \text{ if } J_{s(t+1)}(t+1) = J_{s(t)}(t) \quad (35)$$

Algorithm 1.

Proposed Algorithm to solve Problem (JToba)

```

1: Initialization:
2: Initialize the evaluation and target Q network parameters with  $w$ .
3: Initialize replay memory.
4: for episode  $k \leq 1, 2, \dots, K$  do
5:   if  $\text{mod}(k, 100) == 0$  then
6:     Change the initialization to the current best result:
        $s(t) = s^{\text{init}} = \arg \min_s J_s(t)$ .
7:   end if
8:   Generate a random number  $p$  according to the uniform distribution within  $[0, 1]$ .
9:   if  $p < \varepsilon$  then
10:    Set  $a(t) = \arg \max_a Q^{\text{pre}}(s(t), a; w)$ .
11:   else
12:    Randomly select  $a(t)$  according to a uniform distribution.
13:   end if
14:   Calculate  $J_{s(t)}(t)$  according to (11)
15:   if  $J_{s(t+1)}(t+1) < J_{s(t)}(t)$  then
16:     Set  $r(t) = 1$ 
17:   else if  $J_{s(t+1)}(t+1) > J_{s(t)}(t)$  then
18:     Set  $r(t) = -1$ 
19:   else
20:     Set  $r(t) = 0$ 
21:   end if
22:   Save this memory formed as  $(s(t), a(t), r(t), s(t+1))$  in the replay memory.
23:   Extract a mini-batch of memories from the replay memory.
24:   Calculate the label Q-value  $Q^{\text{lab}}$ :
        $Q^{\text{lab}} = r(t) + \gamma \max_{a'} Q^{\text{tar}}(s(t+1), a'; w')$ .
25:   Use the gradient descent algorithm to minimize  $(Q^{\text{lab}} - Q^{\text{pre}}(s(t), a(t); w))^2$ .
26:   Update the parameters  $w$  of the evaluation network.
27: end for

```

4. Performance evaluation

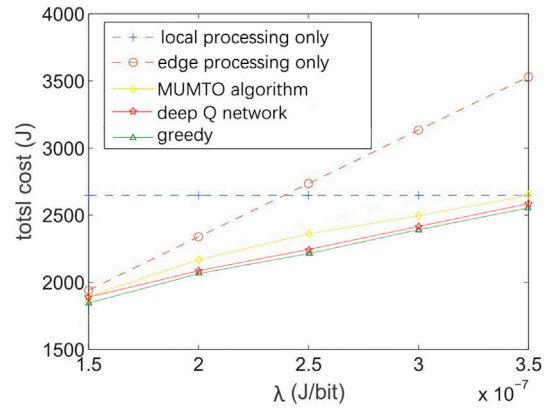
We show the numerical results of our proposed algorithm for solving Problem (JToba) in this section. In particular, we use the TensorFlow to implement DQN. TensorFlow is an open source software library for machine learning algorithms, which has attracted great interests from both academia and industry, with a variety of applications, e.g., speech recognition, play gaming, Google Photos, and web searching [23–26]. In our simulation, we set the number of mobile users $N = 5$ and the number of tasks for each user $M = 4$. In addition, we use the similar parameter-setting as [5], i.e., we set the local computation time of the mobile device as 4.75×10^{-7} s/bit and the processing energy consumption as 3.25×10^{-7} J/bit. We assume that the input data sizes of all tasks are randomly distributed between 10 MB and 30 MB, and the output data sizes are randomly distributed between 1 MB and 3 MB. In the simulations, we set both the uplink and downlink bandwidth limits as 150 Mbps and the mobile device's energy efficiency for both transmitting and receiving data as 1.42×10^{-7} J/bit. The CPU rate of the remote edge server is 10×10^9 cycle/s. We further set $\lambda = 2.5 \times 10^{-7}$ J/bit and $\omega_n = 1$ J/s. Table 2 provides a summary of the detailed parameter-setting.

In Fig. 4, we compare the performance of our proposed DQN based method with other four schemes, i.e., the local processing only scheme, the edge processing only scheme, the greedy scheme, and the MUMTO algorithm proposed in Ref. [5]. The local processing only scheme means that all users' tasks are processed locally. The edge processing only scheme means that the edge server processes all users' tasks. In the greedy scheme, we enumerate all binary offloading decision combinations, and then adopt the one with the best result. However, the greedy scheme is very time-consuming, especially for large-size of users and tasks. In Ref. [5], a MUMTO algorithm has been proposed by using separable semidefinite relaxation.

Table 2

Parameter values used in the simulations.

Parameter	Value	Description
Episode	4000	Number of main cycles
Replay memory size	2000	Size of memory pool
Frequency of learning	5	The frequency to perform the training step
Mini-batch size	32	Number of memories are used for each training step
Learning rate	0.0001	Learning rate of Adamoptimizer
Reward decay	0.9	Emphasis of previous experience
ε -greedy increment	0.005	The growth rate of exploration threshold ε at every training step
Target network update frequency	50	Frequency (or the number of the steps) to update steps the target network is updated
Pre-training steps	200	Number of the memories are stored before the training begins

**Fig. 4.** The total cost under different λ (J/bit).

In Fig. 4, we vary the weight λ used in Eq. (2) to weight the computing delay via the edge server and present the comparison results among these five schemes. Each point in Fig. 4 denotes the averaged result of 100 random realizations of the input/output data sizes. The results show that the DQN algorithm can find a near-optimal solution, i.e., the gap between our DQN algorithm and the greedy scheme is very small. Meanwhile, the results also show that our DQN algorithm outperforms the MUMTO algorithm.

Fig. 5 shows the effects of the mini-batch size on the convergence of our DQN-based algorithm. Mini-batch is used to determine how many experience samples are trained each time. From Fig. 5, we can see that when the mini-batch size is 32 (i.e., 32 records in the replay memory are used to train DQN), the convergence rate is faster when the mini-batch sizes of 64 and 128 are used. Because the smaller the mini-batch size, the steeper the direction of the gradient descent, then the weight of the neural network will be updated faster, i.e., it will converge faster.

Fig. 6 shows the performance of the system's total cost at different learning rates, which is used to adjust the update speed of the weight w in DQN. The convergence process is faster when the learning rate is 0.001 than when the learning rate is 0.0001. However, when the learning rate increases, we suffer from a larger possibility of obtaining a local optimal solution instead of the global optimal solution. Hence, we need to choose an appropriate learning rate according to specific situations.

Fig. 7 shows the performance of the system total cost at different values of λ which is used in Eq. (2). The results show that when λ increases, the system total cost increases. Fig. 8 shows the performance of the system total cost at different values of f^{clo} which is used in Eq. (5). The results show that when f^{clo} increases, the system total cost decreases, which is consistent with the intuition. From all the above simulation results, we observe that the proposed DQN scheme shows a stable convergence performance as a close-to-optimal solution for all the tested parameter-settings.

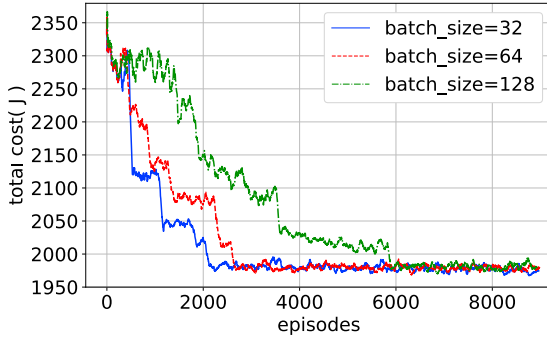


Fig. 5. Convergence performance under different batch sizes.

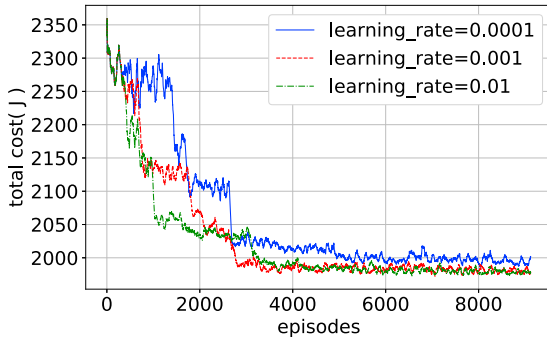


Fig. 6. Convergence performances under different learning rates.

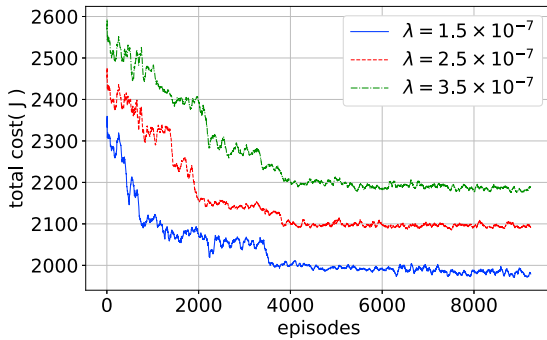


Fig. 7. Convergence performances at different values of λ .

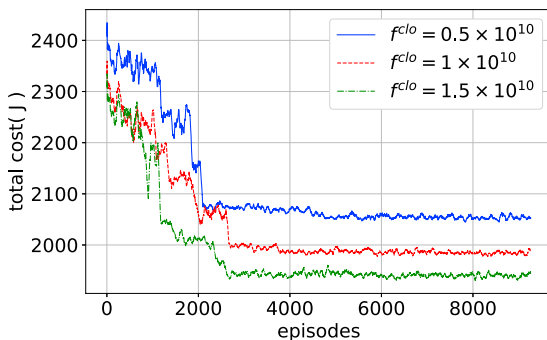


Fig. 8. Convergence performances at different values of f^{clo} .

5. Conclusion

In this work, we investigate the deep reinforcement learning based joint task offloading and bandwidth allocation for multi-user mobile edge computing. We formulate a joint optimization of the task offloading and bandwidth allocation, with the objective of minimizing the overall cost, including the total energy consumption and the delay in finishing the task. Despite its nature of being a mixed binary nonlinear programming, we adopt the idea of DQN and propose an efficient algorithm to find a close-to-optimal solution. Numerical results have validated the accuracy of the proposed DQN-based algorithm and its advantages in performance compared with the existing MUMTO algorithm. For our future work, we will investigate the distributed implementation of our proposed DQN based algorithm, in which the users determine their respective offloading decisions in a distributed manner via learning.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grants No. 61572440 and No. 61502428, and in part by the Zhejiang Provincial Natural Science Foundation of China under Grants No. LR16F010003 and No. LY19F020033.

References

- [1] M. Chiang, T. Zhang, Fog and IoT: an overview of research opportunities, *IEEE Internet Things J.* 3 (6) (2016) 854–864.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, K. Letaief, A survey on mobile edge computing: the communication perspective, *IEEE Commun. Surv. Tutorials* 19 (4) (2017) 2322–2358.
- [3] C. Wang, C. Liang, F. Yu, Q. Chen, L. Tang, Computation offloading and resource allocation in wireless cellular networks with mobile edge computing, *IEEE Trans. Wireless Commun.* 16 (8) (2017) 4924–4938.
- [4] X. Chen, L. Jiao, W. Li, X. Fu, Efficient multi-user computation offloading for mobile-edge cloud computing, *IEEE/ACM Trans. Netw.* 24 (5) (2016) 2795–2808.
- [5] M. Chen, B. Liang, M. Dong, Joint offloading decision and resource allocation for multi-user multi-task mobile cloud, in: 2016 IEEE International Conference on Communications (ICC), Kuala Lumpur, 2016, pp. 1–6.
- [6] O. Munoz, A. Pascual-Iserte, J. Vidal, Optimization of radio and computational resources for energy efficiency in latency-constrained application offloading, *IEEE Trans. Veh. Technol.* 64 (10) (2015) 4738–4755.
- [7] J. Ren, G. Yu, Y. Cai, Y. He, Latency optimization for resource allocation in mobile-edge computation Offloading, *IEEE Trans. Wireless Commun.* 17 (8) (2018) 5506–5519.
- [8] Y. Wang, M. Sheng, X. Wang, L. Wang, J. Li, Mobile-edge computing: partial computation offloading using dynamic voltage scaling, *IEEE Trans. Commun.* 64 (10) (2016) 4268–4282.
- [9] C. You, K. Huang, H. Chae, B. Kim, Energy-efficient resource allocation for mobile-edge computation offloading, *IEEE Trans. Wireless Commun.* 16 (3) (2017) 1397–1411.
- [10] D. Silver, A. Huang, et al., Mastering the game of Go with deep neural networks and tree search, *Nature* 529 (2016) 484–489.
- [11] Y. He, Z. Zhang, et al., Deep-reinforcement-learning-based optimization for cache-enabled opportunistic interference alignment wireless networks, *IEEE Trans. Veh. Technol.* 66 (11) (2017) 10433–10445.
- [12] X. Chen, H. Zhang, et al., Optimized Computation Offloading Performance in Virtual Edge Computing Systems via Deep Reinforcement Learning, 2018 arXiv: 1805.06146.
- [13] K. Cho, Y. Sung, K. Um, A production technique for a Q-table with an influence map for speeding up Q-learning, in: The 2007 International Conference on Intelligent Pervasive Computing, IPC, 2007, pp. 72–75.
- [14] Y. Hirashima, Y. Iiguni, A. Inoue, S. Masuda, Q-learning algorithm using an adaptive-sized Q-table, in: Proceedings of the 38th IEEE Conference on Decision and Control, 1999, pp. 1599–1604.
- [15] H. Zhu, I. Paschalidis, M. Hasselmo, Feature extraction in Q-learning using neural networks, in: 2017 IEEE 56th Annual Conference on Decision and Control (CDC), VIC, Melbourne, 2017, pp. 3330–3335.
- [16] S. Phaniteja, P. Dewangan, P. Guhan, A. Sarkar, K. Krishna, A deep reinforcement learning approach for dynamically stable inverse kinematics of humanoid robots, in: 2017 IEEE International Conference on Robotics and Biomimetics (ROBIO), 2017, pp. 1818–1823.
- [17] A. Sharma, P. Kaushik, Literature survey of statistical, deep and reinforcement learning in natural language processing, in: 2017 International Conference on Computing, Communication and Automation (ICCCA), 2017, pp. 350–354.
- [18] V. Mnih, K. Kavukcuoglu, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.

- [19] S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University press, England, U.K., 2004.
- [20] R. Sutton, A. Barto, *Reinforcement Learning: an Introduction*, The MIT Press, London, U.K., 2018.
- [21] N. Arun, S. Praveen, et al., *Massively Parallel Methods for Deep Reinforcement Learning*, 2015 arXiv:1507.04296.
- [22] L. Ruishan, Z. James, *The Effects of Memory Replay in Reinforcement Learning*, 2017 arXiv:1710.06574.
- [23] M. Abadi, A. Agarwal, et al., *Tensorflow: Large-scale Machine Learning on Heterogeneous Systems*, 2015 arXiv:1603.04467.
- [24] N. Gavai, Y. Jakhade, S. Tribhuvan, R. Bhattad, *MobileNets for flower classification using TensorFlow*, in: 2017 International Conference on Big Data, IoT and Data Science (BID), 2017, pp. 154–158.
- [25] F. Ertam, G. Aydin, *Data classification with deep learning using TensorFlow*, in: 2017 International Conference on Computer Science and Engineering, UBMK, Antalya, 2017, pp. 755–758.
- [26] L. Yuan, Z. Qu, Y. Zhao, H. Zhang, Q. Nian, *A convolutional neural network based on TensorFlow for face recognition*, in: 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), 2017, pp. 525–529.